



Universidad Experimental De Guayana

Vicerrectorado Académico

Coordinación General De Pregrado

Proyecto De Carrera: Ingeniería En Informática

Asignatura: Lenguajes y Compiladores

Lenguajes y Gramáticas Formales

Profesor:

Márquez Félix

Alumnos:

C.I.:

Aguilera Rhixeidys V- 30.851.503

Curbata Emilio V- 28.214.578

Bravo Yvanna V- 28.530.717

Albornoz Ronniel V- 24.889.173

Ciudad Guayana, junio de 2026

Resumen Académico

El presente informe aborda los fundamentos teóricos y la aplicación práctica de los lenguajes y gramáticas formales en el diseño de compiladores. El estudio inicia con el análisis de la relación generativa entre gramáticas y lenguajes mediante el proceso algorítmico de derivación, seguido de la clasificación de su poder computacional a través de la Jerarquía de Chomsky. Para evidenciar la aplicabilidad de estos modelos, se desarrolla un intérprete gráfico basado en *turtle graphics* que procesa cadenas generadas por una Gramática Libre de Contexto (GLC). Esta gramática utiliza un alfabeto bioinformático extendido ($\Sigma = \{a, c, g, t, d, h\}$) para trazar de forma inequívoca figuras geométricas jerárquicas. Asimismo, se enfatiza la importancia de la "higiene de gramáticas" como práctica de ingeniería algorítmica para evitar fallos en los analizadores. Se diagnostican y optimizan tres patologías estructurales: ambigüedad, recursividad por la izquierda y la necesidad de factorización por la izquierda, asegurando así un análisis sintáctico determinístico. Por último, se modela un subconjunto regular de la Notación Portátil de Juegos de Ajedrez (PGN) para movimientos básicos. Se demuestra empíricamente el Teorema de Kleene al diseñar una Expresión Regular y su Autómata Finito Determinístico (AFD) equivalente, consolidando el ciclo completo desde el formalismo matemático hasta la implementación de un reconocedor léxico funcional en Python.

Introducción

En la informática teórica y la construcción de compiladores, el código fuente no se trata como un texto arbitrario, sino como un objeto matemático regido por propiedades demostrables. Para diseñar e implementar analizadores robustos, resulta imperativo comprender en profundidad cómo se generan y reconocen los lenguajes formales. El presente documento explora estas herramientas matemáticas, partiendo desde los conceptos básicos de alfabetos y palabras, hasta llegar a las reglas de producción que estructuran la sintaxis de un lenguaje a través de las gramáticas formales.

A lo largo de este informe, se analiza la Jerarquía de Chomsky como brújula para determinar la capacidad computacional necesaria en el procesamiento de diferentes lenguajes. La teoría se materializa a través de tres casos prácticos fundamentales: el modelado de instrucciones gráficas mediante derivaciones de una Gramática Libre de Contexto para simular el trazo de figuras análogas a estructuras biológicas, la aplicación de técnicas de higiene gramatical para resolver patologías críticas que comprometen la viabilidad operativa de un compilador, y finalmente, la delimitación de un subconjunto del lenguaje de ajedrez PGN. Este último caso ilustra el diseño de una expresión regular y su conversión a un Autómata Finito Determinístico (AFD), pilar esencial del análisis léxico algorítmico.

1. Fundamentos y Jerarquía (Teoría Aplicada)

1.1 Relación Gramática-Lenguaje

La relación entre una gramática y un lenguaje es estrictamente de causalidad matemática: la gramática actúa como el mecanismo generador, y el lenguaje es el producto resultante. Formalmente, una gramática G se define como una tupla compuesta por variables (símbolos no terminales), símbolos de un alfabeto (terminales), un símbolo inicial y un conjunto de reglas de producción. El lenguaje formal, denotado como $L(G)$, es el conjunto infinito o finito de todas las cadenas válidas compuestas únicamente por símbolos terminales que pueden ser producidas al aplicar sistemáticamente las reglas de dicha gramática.

- **Mecanismo de Generación (Derivación):**

El mecanismo de generación se fundamenta en un proceso algorítmico denominado **derivación**.

- Todo proceso comienza con un único "Símbolo Inicial" designado por la gramática.
- A través de iteraciones sucesivas, se inspecciona la cadena actual y se reemplaza cualquier símbolo no terminal presente utilizando una de las reglas de producción disponibles (es decir, el lado izquierdo de una regla se sustituye por su lado derecho correspondiente).
- Este ciclo de sustitución continúa expandiendo o transformando la cadena hasta que ya no existan símbolos no terminales. Cuando la secuencia final está compuesta enteramente por símbolos terminales, se concluye la derivación y la palabra se considera matemáticamente válida y perteneciente al lenguaje.

- **Ejemplo Práctico:**

- **Lenguaje formal deseado:** Cadenas que posean una cantidad idéntica de ceros y unos en orden secuencial ($0^n 1^n$ donde $n \geq 1$).
- **Gramática definida:** * Regla 1 (Recursiva): $S \rightarrow 0S1$
 - Regla 2 (Base): $S \rightarrow 01$
- **Derivación para generar la palabra "000111":** $S \Rightarrow 0S1 \Rightarrow 0(0S1)1 \Rightarrow 00(01)11 \Rightarrow 000111$

1.2 Jerarquía de Chomsky

La Jerarquía de Chomsky clasifica las gramáticas en cuatro niveles concéntricos (del Tipo 0 al Tipo 3). A medida que aumenta el número del tipo, las reglas de producción se vuelven más restrictivas y limitadas, lo que a su vez requiere algoritmos de reconocimiento más sencillos y eficientes.

La notación Backus-Naur Form (BNF) fue concebida históricamente para describir Gramáticas Libres de Contexto (Tipo 2). Para ilustrar los Tipos 0 y 1 mediante BNF, se utiliza una abstracción o "pseudo-BNF" que admite múltiples símbolos en el lado izquierdo de la producción, ilustrando el principio teórico.

- **Tipo 0: Gramáticas Recursivamente Enumerables (Irrestrictas)**

Representan el nivel más poderoso y general de la jerarquía. No poseen ninguna restricción sintáctica en sus reglas de producción; el lado izquierdo o derecho de una regla puede crecer, reducirse o transformarse de cualquier manera. Todo lenguaje que pueda ser computado por una Máquina de Turing teórica pertenece a esta categoría.

- **Ejemplo Práctico (Pseudo-BNF adaptado):**

```
<Variable_A> <Variable_B> ::= <Variable_A> "cadena_terminal" | "fin"
```

- **Tipo 1: Gramáticas Sensibles al Contexto**

En este nivel, la sustitución de una variable por otra secuencia de caracteres depende estrictamente del entorno (contexto) que la rodea. Una regla establece que un no terminal solo muta si tiene símbolos específicos a su izquierda o derecha. Además, la longitud de la cadena generada debe ser siempre mayor o igual a la cadena original (regla de no contracción). Son analizadas por Autómatas Linealmente Acotados.

- **Ejemplo Práctico (Pseudo-BNF adaptado):**

```
"inicio" <Variable_X> "fin" ::= "inicio" "valor_nuevo" "fin"
```

- **Tipo 2: Gramáticas Libres de Contexto (GLC)**

La restricción principal es que el lado izquierdo de cada producción debe estar conformado por un único, y solo un, símbolo no terminal. Esta transformación es independiente de los símbolos adyacentes (es libre de contexto). Poseen la capacidad matemática de "recordar" estructuras anidadas y jerárquicas, por lo que son el núcleo en la construcción de árboles de análisis sintáctico (parsers) de todos los lenguajes de programación. Se implementan computacionalmente mediante Autómatas de Pila.

- **Ejemplo Práctico (BNF Estándar):**

```
<condicional> ::= "if" "(" <expresion> ")" "{" <bloque> "}"
```

- **Tipo 3: Gramáticas Regulares**

Es el nivel más restrictivo y computacionalmente eficiente. Las reglas dictan que un no terminal solo puede generar un único símbolo terminal, o a lo sumo, un símbolo terminal seguido de un único no terminal (gramática regular por la derecha). Su simplicidad impide que validen estructuras anidadas (como paréntesis matemáticos), pero las hace idóneas para el análisis léxico: la clasificación rápida de texto lineal en tokens (identificadores, palabras clave, números). Se procesan utilizando Autómatas Finitos o Expresiones Regulares.

- **Ejemplo Práctico (BNF para gramática regular):**

```
<numero> ::= "9" <digito_restante> | "9"  
<digito_restante> ::= "5" <digito_restante> | "5"
```

2. Derivación y Modelado (Caso Práctico: Genoma)

En bioinformática, el ADN se representa convencionalmente mediante un alfabeto de cuatro nucleótidos: adenina (**a**), citosina (**c**), guanina (**g**) y timina (**t**). Esta analogía resulta de gran utilidad en la teoría de lenguajes formales, ya que permite modelar secuencias biológicas y, por extensión, instrucciones gráficas como cadenas generadas por una gramática.

En el presente caso práctico se reutiliza ese alfabeto $\Sigma = \{a, c, g, t\}$ para codificar comandos de dibujo, ampliándolo con dos terminales adicionales: **d** (giro de 120°) y **h** (hoja), de manera mínima y retro-compatible. Ninguna producción de las variables originales fue modificada; solo se añadieron las variables **D** y **H** con sus respectivas producciones.

2.1. Gramática para Dibujar

Una Gramática Libre de Contexto se define formalmente como la cuádrupla $G = (V, \Sigma, S, P)$, donde V es el conjunto de variables (no terminales), Σ el alfabeto de terminales, S el

símbolo inicial y P el conjunto de producciones. La gramática ampliada utilizada en este caso práctico es la siguiente:

- **Definición formal**

Elemento	Valor
Variables (V)	{ S, F, L, G, D, R, B, H }
Terminales (Σ)	{ a, c, d, g, t, h }
Símbolo inicial	S

- **Producciones (P)**

Variable	Producción
S	F
F	F L F G F D F R F H L G D R H
L	a L a (avance simple o repetido)
G	c (giro 90° a la derecha)
D	d (giro 120° a la derecha — isometría)
R	g B t (rama: push – cuerpo – pop)
B	F B F (cuerpo de rama)
H	h (dibuja una hoja en posición actual)

- **Semántica de los terminales**

La siguiente tabla asigna a cada terminal una acción concreta de dibujo, interpretada mediante el intérprete de cadenas basado en turtle graphics implementado en el código Python:

Símbolo	Acción de dibujo
a	Avanzar una unidad en la dirección actual (trazar segmento)
c	Girar 90° a la derecha

d	Girar 120° a la derecha (isometría / ramas divergentes)
g	Guardar posición y dirección actuales (push — abrir rama)
t	Restaurar la última posición guardada (pop — cerrar rama)
h	Dibujar una hoja en la posición actual, sin desplazar la tortuga

2.2. Intérprete de Cadenas (turtle graphics)

El intérprete recorre caracter por caracter la cadena generada por la gramática y ejecuta la acción correspondiente a cada terminal. Utiliza una **pila** para manejar las ramas (push con **g**, pop con **t**).

La función principal del intérprete (simplificada) es:

```
def interpretar_cadena(cadena, t, paso=60, angulo=90):
    pila = []
    for simbolo in cadena:
        if simbolo == 'a': t.forward(paso)
        elif simbolo == 'c': t.right(angulo) # 90°
        elif simbolo == 'd': t.right(120) # 120°
        elif simbolo == 'g': pila.append((t.pos(), t.heading())) # push
        elif simbolo == 't': pos, h = pila.pop(); t.goto(pos); t.setheading(h)
    # pop
    elif simbolo == 'h': t.dot(8, 'green') # hoja
```

2.3. Cinco Derivaciones Completas

A continuación, se presentan cinco derivaciones paso a paso desde el símbolo inicial **s** hasta una cadena final compuesta exclusivamente por elementos de $\Sigma = \{a, c, d, g, t, h\}$. El símbolo **=>** denota un paso de derivación leftmost (se sustituye el no terminal más a la izquierda en cada paso).

- **Derivación 1 — Cuadrado**

Avanzar y girar 90° cuatro veces consecutivas produce un cuadrado cerrado.

Pasos de derivación (derivación más a la izquierda — leftmost):

```
S
=> F
```

```

=> F L G
=> F L G L G
=> F L G L G L G
=> L G L G L G L G
=> a G L G L G L G
=> a c L G L G L G
=> a c a G L G L G
=> a c a c L G L G
=> a c a c a G L G
=> a c a c a c L G
=> a c a c a c a G
=> a c a c a c a c

```

Cadena final: **acacacac**

Interpretación: Cuatro avances (a) intercalados con cuatro giros de 90° (c) trazan un cuadrado cerrado.

- **Derivación 2 — Árbol con dos ramas y hojas**

Tronco largo (aaaa), dos ramas divergentes giradas 120° (d), cada una con hoja (h) al final.

Pasos de derivación (derivación más a la izquierda — leftmost):

```

S
=> F
=> F L
=> F L L
=> F L L L
=> F L L L D R
=> F L L L D R D R
=> L L L L D R D R
=> a L L L D R D R
=> a a L L D R D R
=> a a a L D R D R
=> a a a a D R D R
=> a a a a d R D R
=> a a a a d g B t D R
=> a a a a d g F t D R
=> a a a a d g L t D R
=> a a a a d g a t D R
=> a a a a d g a H t D R
=> a a a a d g a h t D R
=> a a a a d g a h t d R
=> a a a a d g a h t d g B t
=> a a a a d g a h t d g F t
=> a a a a d g a h t d g L t
=> a a a a d g a h t d g a t
=> a a a a d g a h t d g a H t
=> a a a a d g a h t d g a h t

```

Cadena final: **aaaadgahtdgaht**

Interpretación: Tronco de cuatro avances (aaaa); primer giro de 120° (d), rama que avanza (a) y termina en hoja (h), cierre (t); segundo giro de 120° (d) y segunda rama simétrica con hoja. Resultado: árbol con dos ramas claramente divergentes.

- **Derivación 3 — Triángulo equilátero**

Tres avances separados por giros de 120° (d) trazan un triángulo equilátero cerrado.

Pasos de derivación (derivación más a la izquierda — leftmost):

```
S
=> F
=> F D F
=> F D F D F
=> L D F D F
=> a D F D F
=> a d F D F
=> a d L D F
=> a d a D F
=> a d a d F
=> a d a d L
=> a d a d a
```

Cadena final: adadad

Interpretación: Tres segmentos (a) separados por giros de 120° (d) forman un triángulo equilátero. La suma de los tres giros exteriores de 120° completa los 360° necesarios para cerrar la figura.

- **Derivación 4 — Línea con giro (forma en L invertida)**

Avanzar, girar 90° y avanzar de nuevo produce una figura en forma de L.

Pasos de derivación (derivación más a la izquierda — leftmost):

```
S
=> F
=> F G L
=> L G L
=> a G L
=> a c L
=> a c a
```

Cadena final: *aca*

Interpretación: Un avance (a), un giro de 90° (c) y un segundo avance (a) producen una figura en forma de L invertida.

- **Derivación 5 — Escalera (tres peldaños)**

Cada peldaño se forma con: sube (a), avanza a la derecha (c→a), regresa al Norte (ccc).

Repetido tres veces produce una escalera ascendente.

Pasos de derivación (derivación más a la izquierda — leftmost):

```
S
=> F
=> F G
=> F G L
=> F G L G
=> F G L G G
=> F G L G G G
=> F G L G G G L
=> L G L G G G L G G G L G G G
=> a G L G G G L G G G L G G G
=> a c L G G G L G G G L G G G
=> a c a G G G L G G G L G G G
=> a c a c G G L G G G L G G G
=> a c a c c G L G G G L G G G
=> a c a c c c L G G G L G G G
=> a c a c c c a G G G L G G G
=> ... (patrón se repite para los tres peldaños)
=> a c a c c c a c a c c c a c a c
```

Cadena final: *acacccacacccacac*

Interpretación: Tres repeticiones del patrón (avanza, gira derecha, avanza, vuelve al Norte con tres giros) producen una escalera de tres peldaños que asciende de izquierda a derecha.

2.4. Tabla Resumen de Derivaciones

Nº	Figura	Cadena final	Longitud
1	Cuadrado	acacacac	8
2	Árbol con dos ramas y hojas	aaaadgahtdgaht	14
3	Triángulo equilátero	adadad	6
4	Línea con giro (forma en L invertida)	aca	3
5	Escalera (tres peldaños)	acacccacacccacac	16

El ejercicio demuestra que una Gramática Libre de Contexto definida sobre una extensión mínima del alfabeto del ADN ($\Sigma = \{a, c, d, g, t, h\}$) es capaz de generar, de forma verificable y sin ambigüedad operativa, cadenas con significado gráfico preciso: un cuadrado, un árbol con ramas y hojas, un triángulo equilátero, una línea con giro y una escalera de tres peldaños.

Cada producción tiene correspondencia directa con una acción del intérprete Python, lo que evidencia la potencia del formalismo gramatical para describir sistemas generativos. La implementación en Python con `turtle` permite visualizar en tiempo real cada cadena derivada, convirtiendo la gramática en una herramienta pedagógica tangible. La identificación de estas estructuras sienta las bases para la construcción de analizadores léxicos y sintácticos que procesen dichas cadenas de forma automática (Aho et al., 2007).

3. Higiene y Optimización de Gramáticas

En el ámbito del diseño de compiladores, el modelo matemático que subyace a la validación de la sintaxis es fundamental. Como se establece en los fundamentos teóricos del procesamiento de lenguajes, una gramática mal diseñada puede conducir a un compilador a estados de fallo crítico, tales como bucles infinitos, rechazo de código fuente válido o la generación de árboles de sintaxis contradictorios. Por tanto, la "higiene de gramáticas" trasciende el mero ejercicio académico para consolidarse como una práctica de ingeniería algorítmica esencial. A continuación, se diagnostican tres patologías gramaticales comunes y se aplican sus respectivos métodos formales de optimización.

3.1 Patologías de las Gramáticas y Casos Prácticos

a) Gramática Ambigua

Desde la perspectiva de la teoría de lenguajes, una gramática es ambigua si, y solo si, es capaz de generar más de un árbol de derivación sintáctica (o árbol de análisis) para una misma

cadena de entrada. En la construcción de intérpretes y compiladores, esto representa un error semántico grave, ya que el evaluador no tendría certeza matemática sobre qué orden de precedencia aplicar a las instrucciones, derivando en interpretaciones lógicas contradictorias.

- **Caso Práctico: Evaluación de Expresiones Aritméticas**

Sea la siguiente gramática libre de contexto que modela operaciones matemáticas básicas sin jerarquía de operadores: $E \rightarrow E + E \mid E * E \mid id$

Dada la cadena de entrada " $id + id * id$ ", el analizador puede construir dos árboles de derivación estructuralmente distintos:

1. **Árbol de Derivación A (Asociatividad por la izquierda en la suma):**

- Raíz: $E \rightarrow E * E$
- El nodo E izquierdo deriva en: $E \rightarrow E + E$
- Los nodos resultantes derivan a los tokens terminales id .
- *Resultado lógico:* La máquina interpretará la operación como $(id + id) * id$.

2. **Árbol de Derivación B (Asociatividad por la derecha en la multiplicación):**

- Raíz: $E \rightarrow E + E$
- El nodo E derecho deriva en: $E \rightarrow E * E$
- Los nodos resultantes derivan a los tokens terminales id .
- *Resultado lógico:* La máquina interpretará la operación como $id + (id * id)$.

La existencia empírica de estos dos árboles para una única cadena de entrada demuestra formalmente la ambigüedad de la gramática propuesta.

b) Recursividad por la Izquierda

La recursividad por la izquierda es una patología estructural donde un símbolo no terminal contiene una producción que deriva en una cadena que comienza exactamente con ese mismo

símbolo no terminal ($A \rightarrow A\alpha$). En la implementación de analizadores sintácticos descendentes (como los parsers predictivos *Top-Down*), esta estructura provoca un desbordamiento de pila (*Stack Overflow*) al inducir al compilador a un bucle infinito de llamadas recursivas sin consumir ningún token de entrada.

- **Caso Práctico: Lista Continua de Sumas**

Sea la gramática recursiva por la izquierda: $E \rightarrow E + T \mid T$

- **Algoritmo de eliminación paso a paso:**

La teoría de compiladores establece que una producción del tipo $A \rightarrow A\alpha \mid \beta$ debe transformarse introduciendo un nuevo símbolo no terminal auxiliar (A').

1. **Identificación de términos:** Se define el símbolo base $\beta = T$ y el término recursivo $\alpha = + T$.
2. **Sustitución de la regla original:** Se reescribe el no terminal E para que inicie con el símbolo base β seguido del nuevo no terminal E' .

$$E \rightarrow T E'$$

3. **Definición de la recursividad por la derecha:** Se define el comportamiento de E' para procesar el resto de la cadena, añadiendo la transición a ϵ (épsilon, cadena vacía) para garantizar la terminación del algoritmo.

$$E' \rightarrow + T E' \mid \epsilon$$

- **Gramática resultante optimizada (Libre de bucles):**

$$E \rightarrow T E'$$

$$E' \rightarrow + T E' \mid \epsilon$$

c) Factorización por la Izquierda

Esta técnica de optimización es imperativa cuando múltiples producciones de un mismo símbolo no terminal comparten un prefijo idéntico. Si el parser predictivo evalúa el primer token y coincide con varias rutas, el autómata pierde su cualidad determinística, requiriendo complejos e ineficientes algoritmos de retroceso (*backtracking*).

- **Caso Práctico: Sentencia de Control de Flujo (IF-THEN-ELSE)**

Considérese la siguiente gramática clásica: $S \rightarrow \text{if } E \text{ then } S \mid \text{if } E \text{ then } S \text{ else } S \mid a$

Las dos primeras producciones comparten el prefijo "if E then S".

- **Proceso de Factorización:**

Para optimizar una estructura $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ se extrae el factor común α y se delega la diferenciación de los sufijos a un nuevo nivel jerárquico.

1. **Extracción del factor común:** $\alpha = \text{if } E \text{ then } S$.
2. **Agrupación de sufijos diferentes:** $\beta_1 = \varepsilon(\text{vacío})$ y $\beta_2 = \text{else } S$.
3. **Construcción de las nuevas producciones:** Se asigna el prefijo común a s concatenado con un nuevo no terminal s' .

- **Gramática resultante optimizada:**

$S \rightarrow \text{if } E \text{ then } S S' \mid a$

$S' \rightarrow \text{else } S \mid \varepsilon$

Esta refactorización asegura que el compilador procese el condicional linealmente, evaluando la existencia del bloque `else` solo después de haber garantizado la sintaxis de la estructura principal.

4. De la Expresión al Autómata (Caso Práctico: Ajedrez).

4.1. Notación PGN y planteamiento del autómata

Una partida en formato PGN se compone de dos secciones: un encabezado de *tag pairs* con metadatos por ejemplo `[Event "..."]` o `[Date "..."]`— y el *movetext*, que contiene la secuencia de jugadas y el resultado final, como en `1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 1-0` (Edwards, 1994).

Cada jugada se codifica en SAN según las reglas resumidas en la Tabla 1.

Tabla 1. Componentes de una jugada en Notación Algebraica Estándar

Tipo de jugada	Ejemplo	Descripción
Movimiento de peón	e4	Casilla destino (columna + fila)
Movimiento de pieza	Nf3	Letra de pieza + casilla destino
Captura	exd5, Nxe4	El símbolo x denota captura
Enroque corto	O-O	Enroque del flanco de rey
Enroque largo	O-O-O	Enroque del flanco de dama
Jaque / Jaque mate	Qh5+, Qh7#	Sufijos + (jaque) y # (mate)

Las letras de pieza corresponden a sus iniciales en inglés: **K** (Rey), **Q** (Dama), **R** (Torre), **B** (Alfil) y **N** (Caballo); el peón, por ser la pieza más frecuente, se anota sin letra (Edwards, 1994).

Es importante precisar el **alcance** del autómata. El lenguaje PGN completo **no es un lenguaje regular**: admite comentarios y variantes anidadas —mediante llaves { } y paréntesis () que pueden contener, a su vez, otras variantes—, estructura que exige memoria de pila y, por tanto, una Gramática Libre de Contexto (Tipo 2 en la Jerarquía de Chomsky). En cambio, una **jugada SAN individual** sí

constituye un lenguaje regular (Tipo 3), pues carece de anidamiento y puede reconocerse con memoria finita (Hopcroft et al., 2007; Sipser, 2013). Por esta razón se delimita un subconjunto regular, definido formalmente en 2.4.2, que el AFD reconoce de manera completa y eficiente.

4.2. Definición del subconjunto simplificado del PGN

Se define el lenguaje regular **L** como el conjunto de cadenas que representan una jugada básica válida en SAN. La Tabla 2 enumera los patrones admitidos.

Tabla 2. *Patrones del subconjunto L*

#	Patrón	Ejemplos válidos
1	Movimiento de peón	e4, d5, a6
2	Captura de peón	exd5, gxf6
3	Movimiento de pieza	Nf3, Bb5, Qd1, Kg1
4	Captura de pieza	Nxe4, Rxe5
5	Enroque corto o largo	O-O, O-O-O
6	Cualquier patrón anterior con sufijo de jaque o mate	Qh5+, Qh7#, O-O+

Con el fin de preservar la regularidad del lenguaje y la claridad del modelo, se **excluyen** del subconjunto los siguientes elementos de la SAN completa: la desambiguación de origen (p. ej. Nbd2, R1e2), la coronación de peón (p. ej. e8=Q), las anotaciones de evaluación (!, ?), así como los comentarios, variantes y *tag pairs* del encabezado.

El **alfabeto** Σ sobre el que se define L es:

$$\Sigma = \{ K, Q, R, B, N, a, b, c, d, e, f, g, h, 1, 2, 3, 4, 5, 6, 7, 8, x, O, -, +, \# \}$$

Para efectos de la función de transición, estos símbolos se agrupan en **categorías**: **P** (pieza), **F** (columna o *file*), **R** (fila o *rank*), **x** (captura), **O** (enroque), **-** (guion) y **J** (jaque o mate, es decir **+** o **#**). Esta agrupación reduce el número de columnas de la tabla de transición sin alterar el lenguaje reconocido.

4.3. Expresión Regular y Autómata Finito Determinístico equivalente

- **Expresión Regular**

El subconjunto L queda descrito por la siguiente expresión regular:

$$\wedge ((?:[KQRBN]x? \mid [a-h]x)? [a-h][1-8] \mid O-O(?:-O)?) [+ \#]? \$$$

La Tabla 3 descompone su semántica.

Tabla 3. *Análisis de la expresión regular*

Fragmento	Significado
<code>[KQRBN]x?</code>	Una pieza, con captura opcional (N, Nx)
<code>[a-h]x</code>	Columna de origen del peón seguida de captura (ex)
<code>(?:[KQRBN]x? \mid [a-h]x)?</code>	Prefijo opcional; su ausencia indica un movimiento simple de peón
<code>[a-h][1-8]</code>	Casilla destino obligatoria (columna + fila)
<code>O-O(?:-O)?</code>	Enroque corto (O-O) o largo (O-O-O)
<code>[+ \#]?</code>	Sufijo opcional de jaque (+) o mate (#)

Autómata Finito Determinístico

El AFD equivalente se define formalmente como la quintupla $M = (Q, \Sigma, \delta, q_0, F)$, donde:

$$Q = \{ q_0, q_P, q_{pf}, q_f, q_r, q_A, q_{O1}, q_{O2}, q_{C1}, q_{O3}, q_{C2}, q_S, q_{dead} \}$$

Σ es el alfabeto definido en 2.4.2.

$q_0 = q_0$ (estado inicial).

$F = \{ q_A, q_{C1}, q_{C2}, q_S \}$ (estados de aceptación).

δ es la función de transición especificada en la Tabla 4.

Tabla 4. Función de transición δ (las celdas vacías conducen al estado muerto q_{dead})

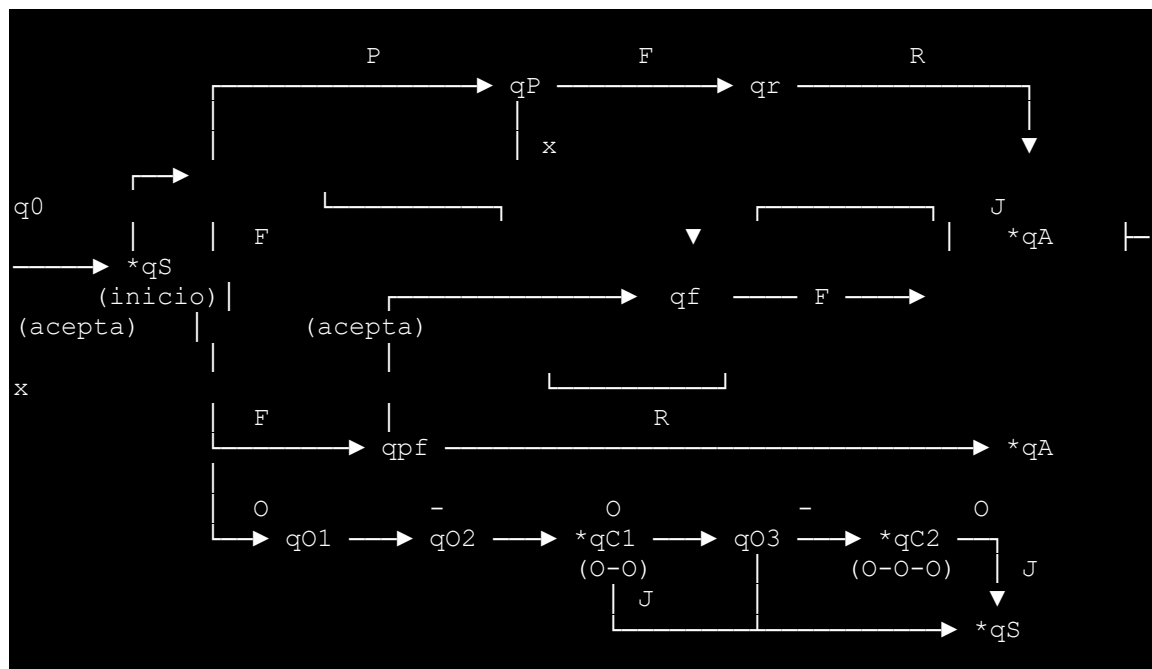
Estado \ Categoría	P	F	R	x	O	-	J
$\rightarrow q_0$	q_P	q_{pf}	—	—	q_{O1}	—	—
q_P	—	q_r	—	q_f	—	—	—
q_{pf}	—	—	q_A	q_f	—	—	—
q_f	—	q_r	—	—	—	—	—
q_r	—	—	q_A	—	—	—	—
* q_A	—	—	—	—	—	—	q_S
q_{O1}	—	—	—	—	—	q_{O2}	—
q_{O2}	—	—	—	—	q_{C1}	—	—
* q_{C1}	—	—	—	—	—	q_{O3}	q_S
q_{O3}	—	—	—	—	q_{C2}	—	—
* q_{C2}	—	—	—	—	—	—	q_S
* q_S	—	—	—	—	—	—	—

Nota. El símbolo $\rightarrow$ identifica el estado inicial y el asterisco (*) los estados de aceptación. El estado **q_{dead}** es un estado trampa (*trap state*): toda

transición no definida conduce a él, y una vez alcanzado no se abandona, lo que modela formalmente el rechazo de la cadena y vuelve a δ una función **total** (Sipser, 2013).

La Figura 1 representa gráficamente el autómata.

Figura 1. Diagrama de estados del AFD (las transiciones a q_{dead} se omiten por convención)



- **Equivalencia y verificación**

La equivalencia entre ambos modelos no es casual, sino una consecuencia del **Teorema de Kleene**, según el cual la clase de lenguajes descritos por expresiones regulares coincide exactamente con la clase de lenguajes reconocidos por autómatas finitos (Kleene, 1956; Hopcroft et al., 2007). Para corroborarlo empíricamente, el reconocedor implementado evalúa un conjunto de 27 cadenas de prueba —15 válidas y 12 inválidas— tanto con el AFD (recorriendo su tabla de

transición) como con la expresión regular, y verifica que ambos veredictos **coinciden en la totalidad de los casos (0 discrepancias)**. Este resultado confirma que el AFD construido reconoce precisamente el lenguaje L descrito por la expresión regular.

- **Ejecución del reconocedor (4.1)**

El autómata se implementó en Python como una tabla de transición explícita, de modo que el código refleja directamente la definición formal de δ . Al procesar una cadena, el programa emite la traza del recorrido estado por estado; por ejemplo, para la captura `Nxe4`:

```
q0 --N(P)--> qP      (leída una pieza)
qP --x(x)--> qf      (esperando columna destino tras captura)
qf --e(F)--> qr      (esperando fila destino)
qr --4(R)--> qA      ACEPTA: jugada completa
```

El reconocedor opera además sobre líneas de *movetext*, separando los números de jugada y el resultado, y validando cada jugada con el AFD; de este modo, sobre una secuencia como `1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 1-0` reconoce correctamente la totalidad de las jugadas.

El caso práctico evidencia el ciclo completo *expresión regular* $\rightarrow$ *autómata finito* $\rightarrow$ *implementación*, que constituye el fundamento del análisis léxico en la construcción de compiladores (Aho et al., 2007). La delimitación de un subconjunto regular del PGN ilustra, asimismo, una decisión de ingeniería esencial: reconocer los límites expresivos de cada nivel de la Jerarquía de Chomsky permite seleccionar el modelo computacional mínimo suficiente —un AFD, de memoria finita— evitando el sobre costo de un analizador sintáctico basado en una gramática libre de contexto cuando el problema no lo requiere.

Conclusión

El desarrollo de los casos prácticos presentados en este informe evidencia que el estudio de los lenguajes y gramáticas formales trasciende el ejercicio académico para constituirse como el cimiento de la ingeniería de compiladores. Se demostró que las gramáticas poseen un poder generativo capaz de mapear símbolos finitos hacia sistemas de representación complejos, tal como se comprobó en la generación exitosa de figuras geométricas a partir de un alfabeto genómico extendido.

De igual forma, se validó que la higiene de las gramáticas es una condición técnica ineludible. Ignorar patologías como la ambigüedad, que genera árboles contradictorios, o la recursividad por la izquierda, que induce desbordamientos de pila, deriva irremediablemente en analizadores ineficientes o incapaces de procesar el código. Por otra parte, la implementación del reconocedor léxico para el lenguaje de ajedrez PGN confirmó en la práctica el Teorema de Kleene y subrayó una lección crucial de diseño: conocer los límites impuestos por la Jerarquía de Chomsky permite a los ingenieros seleccionar el modelo computacional mínimo suficiente para una tarea. En definitiva, el dominio de la abstracción formal, la optimización gramatical y la construcción de autómatas dota al profesional de las herramientas necesarias para abandonar el rol de consumidor de software y asumir la arquitectura de los lenguajes de programación modernos.

Referencias

- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compiladores: Principios, técnicas y herramientas* (2.^a ed.). Pearson Educación.
- Chomsky, N. (1956). Three models for the description of language. *IRE Transactions on Information Theory*, 2(3), 113-124.
- Hopcroft, J. E., Motwani, R., y Ullman, J. D. (2007). *Introduction to Automata Theory, Languages, and Computation* (3.^a ed.). Addison-Wesley.
- Sipser, M. (2013). *Introduction to the Theory of Computation* (3.^a ed.). Cengage Learning.